

WAB

Provadis School of International Management and Technology

Examining the Feasibility of Machine Learning-Based LogP Prediction on an ESP32 Microcontroller

A Proof-of-Concept Implementation and Performance Analysis

Rubin Chempananickal James

rubin.chempananickal-james@stud-provadis-hochschule.de

Matriculation Number: D876

Department: Information Technology

Module: Embedded Systems and Software

Reviewer: Prof. Dr. Eric Hutter

March 29, 2026

Abstract

Contents

Abstract	II
1 Introduction	1
1.1 Background	1
1.2 Motivation	1
1.3 Research Question	2
2 Methods	3
2.1 Dataset	3
2.2 Design	3
2.3 ESP32 Deployment	4
3 Results	5
4 Discussion	6
5 Conclusion	7
AI Declaration	iii
Declaration of Authorship	vi

List of Figures

1.1	A QR Code encoding the ECFP4 fingerprint of aspirin.	2
2.1	Training and validation loss and MAE curves over the 12 epochs.	4

List of Tables

Glossary

ESP32 Espressif Systems’ low-cost, low-power system on a chip (SoC) with integrated Wi-Fi and Bluetooth capabilities. 2, 4

fingerprint A binary vector representation of a molecule, where each bit represents the presence or absence of a particular substructure or feature in the molecule. 2

lipophilicity The chemical property of being fat-soluble, often quantified by the octanol-water partition coefficient ($\log P$). 1

$\log P$ Logarithm of the octanol-water partition coefficient. A measure of a compound’s hydrophobicity, or how well it dissolves in fats versus water. 1–3

QR code Quick Response Code, a type of matrix barcode (or two-dimensional barcode) that can store information such as a URL, text, or other data. 2

ReLU Rectified Linear Unit, a type of activation function used in neural networks that outputs the input directly if it is positive, otherwise, it outputs zero. 3

SMILES Simplified Molecular Input Line Entry System, a notation that allows a user to represent a chemical structure as a string of ASCII characters. 2, 3

TFLite TensorFlow Lite, a framework for deploying machine learning models on mobile and embedded devices. 4

ZINC20 ZINC Is Not Commercial (2020), a free database of commercially available compounds for virtual screening. 3

1 Introduction

1.1 Background

The octanol-water partition coefficient ($\log P$) measures how well a compound dissolves in octanol versus water. It is a fundamental physicochemical property that quantifies the lipophilicity of a molecule.

The logarithm of partition coefficient ($\log P$) between n-octanol and water is a critical property related to the physicochemical and physiological properties of a drug molecule, such as absorption, distribution, metabolism, excretion and toxicity (ADMET) and target protein binding^{1,2,3,4}. Thus it is frequently used in describing the druggability of a pharmaceutical molecule. The first $\log P$ calculation method was developed by Fujita et al. in 1960s^{5,6}. Early on, the structural information of receptor is not easy to obtain, medicinal chemists frequently used quantitative structure-activity relationship (QSAR) to correlate the binding energy of a ligand with its physicochemical characteristics, and $\log P$ was regarded as a key descriptor measuring a drug’s hydrophobicity. Nowadays, Lipinski’s rule of five^{7,8}, published in the 1990s, has been widely used as simple rules to determine the druglikeness of a pharmaceutical molecule. $\log P$ smaller than 5 is one of the four rules in Lipinski’s rule of 5.⁹

1.2 Motivation

To date, there have been many computational methods developed to predict $\log P$ values, including fragment based approaches⁵ free energy based approaches⁹, and recently, machine learning based approaches^{10,11}.

The first two categories of methods are far too computationally intensive to be deployed on a microcontroller. The machine learning based approaches could potentially be feasible, if a neural network could be trained to predict $\log P$ values and translated into a microcontroller compatible format using LiteRT¹² (formerly TensorFlow Lite).

The reason for exploring the ability to predict $\log P$ on a microcontroller is to enable real-time, on-site predictions in various applications, such as environmental monitoring or portable drug discovery tools. By offloading the prediction task away from a host computer and onto a microcontroller, one can potentially reduce latency in systems that require immediate response, such as a microfluidic dispensing device that needs to make

decisions based on the $\log P$ of a compound in real-time.

Since the molecular fingerprint is a bit vector that could be encoded as a QR code (see Figure 1.1 for an example), the microcontroller could also be used in scenarios where a user can scan said code printed on the sample in a laboratory or field setting with a portable device and receive an immediate prediction of the $\log P$ value, and if this approach seems feasible and accurate enough, it could be extended to predicting other properties as well, without needing to access a cloud-based service or a powerful computer.



Figure 1.1: A QR Code encoding the ECFP4 fingerprint of aspirin.

Also, by decoupling the fingerprint generation (a deterministic process on canonical SMILES strings) and the prediction (a non-deterministic process owing to the nature of machine learning models), and by binding the prediction to a specific piece of hardware (a "black box", as it were), one can make the entire system more modular and easier to debug. As an added bonus, the host computer wouldn't need to have any machine learning libraries installed.

1.3 Research Question

The main research question of this project is: How accurately can a quantized machine learning model predict $\log P$ values based on its ECFP4 fingerprint when deployed on an ESP32 microcontroller?

2 Methods

2.1 Dataset

The base dataset used in this project is a subset of the ZINC20 dataset¹³, which contains 249455 commercially available drug-like molecules and their corresponding logP values. Specifically, this subset was extracted at random from the entire ZINC20 dataset by Gómez-Bombarelli et al.¹⁴.

The logP values within the ZINC20 dataset were calculated using Molinspiration’s MiLogP algorithm, which calculates logP based on group contributions¹⁵. Due to the limited availability of large datasets of experimentally confirmed logP values for drug-like molecules, and due to the limited scope of this project, this computed logP will be treated as the ground truth for the purposes of training and evaluating the machine learning model.

2.2 Design

Python¹⁶ version 3.12.13 served as the primary programming language for this project. The entire project was conducted on a laptop running Windows 11 25H2.

The dataset was randomly split into training (80%), validation (10%), and test (10%) sets, with the random seed set to 42 to ensure reproducibility.

The input features for the model are 2048-bit Morgan fingerprints (radius 2) generated from the SMILES representations of molecules. The SMILES were canonicalized and the fingerprint bit vectors were generated using the RDKit¹⁷ library in Python.

The machine learning model is a feedforward Deep Neural Network (DNN) with a single hidden layer of 128 units, using a ReLU activation function, followed by a linear output layer that predicts the logP value. Keras¹⁸ was used as the high-level API for building and training the neural network, while TensorFlow¹⁹ served as the backend. Model training was performed fully on the CPU using the Adam²⁰ optimizer and mean squared error (MSE) as the loss function.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$
 where y_i is the true logP value and $\hat{y}_i$ is the predicted logP value for the i -th sample.

Early stopping was employed to prevent overfitting, monitoring the validation loss and

restoring the best weights. This meant that with a batch size of 256 and the TensorFlow random seed set to 42, the training converged after 12 out of a maximum of 20 epochs (see Figure 2.1).

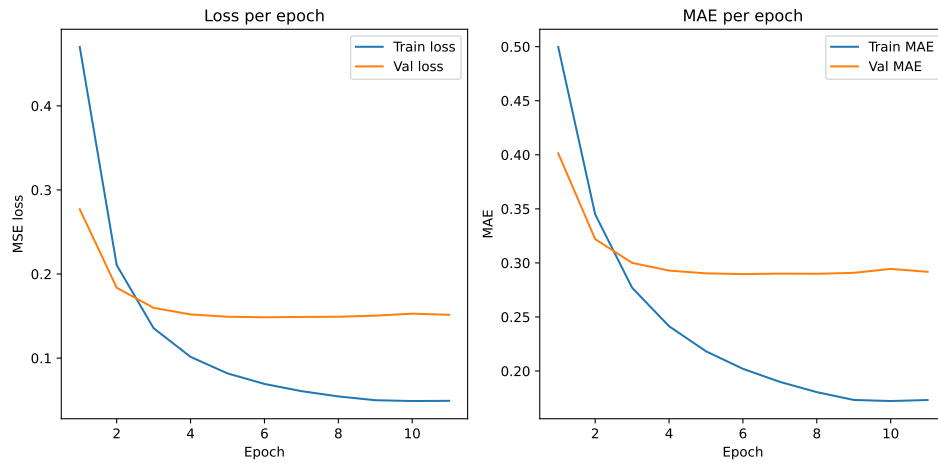


Figure 2.1: Training and validation loss and MAE curves over the 12 epochs.

2.3 ESP32 Deployment

To reduce the model size, the final model was quantized to 8-bit integers using TFLite’s post-training quantization method. The resulting flatbuffer was saved as a .tflite file.

This .tflite file was then converted to a C array using a python tool that emulates the POSIX ‘xxd’ command, and the resulting .cc and .h files, along with an API script, were built and flashed onto an ESP32 microcontroller using ESP-IDF version 5.5.3.

All the necessary code files, along with Visual Studio Code specific instructions for setting up the ESP-IDF environment, building the project, and flashing the firmware, can be found in the supplementary materials.

ESP32 specifications²¹:

- Model: ESP32-WROOM-32D
- CPU: Dual-core Xtensa LX6 microprocessor, up to 240 MHz
- RAM: 520 KB SRAM
- Flash Storage: 4 MB

3 Results

4 Discussion

5 Conclusion

Bibliography

- [1] Jason D. Hughes et al. „Physiochemical drug properties associated with in vivo toxicological outcomes“. In: *Bioorganic & Medicinal Chemistry Letters* 18.17 (2008), pp. 4872–4875. ISSN: 0960-894X. DOI: <https://doi.org/10.1016/j.bmcl.2008.07.071>. URL: <https://www.sciencedirect.com/science/article/pii/S0960894X08008500>.
- [2] Michael J Waring. „Defining optimum lipophilicity and molecular weight ranges for drug candidates—molecular weight dependent lower log D limits based on permeability“. In: *Bioorganic & Medicinal Chemistry Letters* 19.10 (2009), pp. 2844–2851.
- [3] M Paul Gleeson. „Generation of a set of simple, interpretable ADMET rules of thumb“. In: *Journal of Medicinal Chemistry* 51.4 (2008), pp. 817–834.
- [4] Ted W Johnson, Klaus R Dress, and Martin Edwards. „Using the Golden Triangle to optimize clearance and oral absorption“. In: *Bioorganic & Medicinal Chemistry Letters* 19.19 (2009), pp. 5560–5564.
- [5] Toshio Fujita, Junkichi Iwasa, and Corwin Hansch. „A new substituent constant, π , derived from partition coefficients“. In: *Journal of the American Chemical Society* 86.23 (1964), pp. 5175–5180.
- [6] Junkichi Iwasa, Toshio Fujita, and Corwin Hansch. „Substituent constants for aliphatic functions obtained from partition coefficients“. In: *Journal of Medicinal Chemistry* 8.2 (1965), pp. 150–153.
- [7] CA Lipinski et al. „Polar molecular surface properties predict the intestinal absorption of drugs in humans“. In: *Adv. Drug Delivery Rev.* 46.1–3 (2001), pp. 3–26.
- [8] Christopher A Lipinski. „Lead-and drug-like compounds: the rule-of-five revolution“. In: *Drug discovery today: Technologies* 1.4 (2004), pp. 337–341.
- [9] Yuchen Sun et al. „Development and test of highly accurate endpoint free energy methods. 2: Prediction of logarithm of n-octanol-water partition coefficient (logP) for druglike molecules using MM-PBSA method“. In: *Journal of Computational Chemistry* 44.13 (2023), pp. 1300–1311. DOI: <https://doi.org/10.1002/jcc.27086>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/jcc.27086>. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/jcc.27086>.

- [10] Liudmyla N. Ognichenko et al. „QSPR Prediction of Lipophilicity for Organic Compounds Using Random Forest Technique on the Basis of Simplex Representation of Molecular Structure“. In: *Molecular Informatics* 31.3-4 (2012), pp. 273–280. DOI: <https://doi.org/10.1002/minf.201100102>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/minf.201100102>. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/minf.201100102>.
- [11] Juda Baikété, Alhadji Malloum, and Jeanet Conradie. „Comparative study of machine learning methods for accurate prediction of logP and pKb“. In: *Artificial Intelligence Chemistry* 4.1 (2026), p. 100101. ISSN: 2949-7477. DOI: <https://doi.org/10.1016/j.aichem.2025.100101>. URL: <https://www.sciencedirect.com/science/article/pii/S2949747725000181>.
- [12] *LiteRT is Google’s on-device framework for high-performance ML & GenAI deployment on edge platforms*. URL: <https://ai.google.dev/edge/litert> (visited on 03/29/2026).
- [13] John J Irwin et al. „ZINC: a free tool to discover chemistry for biology“. In: *Journal of chemical information and modeling* 52.7 (2012), pp. 1757–1768.
- [14] Rafael Gómez-Bombarelli et al. „Automatic chemical design using a data-driven continuous representation of molecules“. In: *ACS central science* 4.2 (2018), pp. 268–276.
- [15] *Molinspiration: logP - octanol-water partition coefficient*. URL: <https://www.molinspiration.com/services/logp.html> (visited on 03/29/2026).
- [16] Guido Van Rossum and Fred L. Drake. *Python 3 Reference Manual*. Scotts Valley, CA: CreateSpace, 2009. ISBN: 1441412697.
- [17] *RDKit: Open-source cheminformatics software*. DOI: <https://doi.org/10.5281/zenodo.591637>. URL: <https://www.rdkit.org/> (visited on 03/29/2026).
- [18] François Chollet et al. *Keras*. <https://keras.io>. 2015.
- [19] Martín Abadi et al. *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. Software available from tensorflow.org. 2015. URL: <https://www.tensorflow.org/>.
- [20] Diederik Kingma and Jimmy Ba. „Adam: A Method for Stochastic Optimization“. In: *International Conference on Learning Representations* (Dec. 2014).
- [21] *ESP32-WROOM-32D & ESP32-WROOM-32U Datasheet*. URL: https://documentation.espressif.com/esp32-wroom-32d_esp32-wroom-32u_datasheet_en.pdf (visited on 03/29/2026).

AI Declaration

The usage of AI tools within this project is documented here. I solemnly declare that I have documented all interactions with AI tools, including the prompts used and the outputs received.

System	Prompt	Usage
GitHub Copilot 1	Asked to delete the existing project and provide a simple LaTeX template with a shared preamble, a Chapters subdirectory, TOC, glossary, abbreviations, and Roman numerals for non-content pages.	Template structure and LaTeX setup provided
GitHub Copilot 2	Requested additional title page lines (WAB header, reviewer, module) for the main document.	Title page metadata and layout updated
GitHub Copilot 3	Requested uppercase Roman numerals for front matter and lowercase Roman numerals for back matter page numbering.	Page numbering adjusted in main and exposee
GitHub Copilot 4	Requested exposee title page to include shared info fields (WAB header, department, module, reviewer).	Exposee title page updated to include shared metadata
GitHub Copilot 5	Requested adding the provadis-hochschule.pdf logo to the top right corner of both main and exposee title pages.	Logo added to top right of both title pages
GitHub Copilot 6	Reported that Glossary and Abbreviations sections were missing from compiled output.	Added example <code>\newacronym</code> entries to abbreviations.tex so the Abbreviations section displays properly
GitHub Copilot 7	Reported that glossary section still not appearing in final PDF.	Updated settings.tex to include <code>automake</code> option in glossaries package to enable automatic glossary generation
GitHub Copilot 8	Asked for boilerplate to train a TFLite model for ESP32 to estimate logP from SMILES.	Provided a Python training script (RDKit + Keras) with TFLite export
GitHub Copilot 9	Asked how to run inference on an ESP32 with the trained model.	Provided a high-level TFLite Micro deployment workflow for ESP32
GitHub Copilot 10	Asked for a beginner-friendly README in Code/ with step-by-step setup and usage.	Added a novice-oriented Code/README.md with installation, training, and ESP32 notes

System	Prompt	Usage
GitHub Copilot 11	Asked for a TFLite Micro skeleton and beginner ESP32 setup steps.	Added <code>Code/tflite_micro_skeleton.cpp</code> and expanded <code>Code/README.md</code> with ESP32 instructions
GitHub Copilot 12	Reported the TFLite Micro skeleton was incomplete.	Completed the skeleton with quantized support, Arduino setup, and a runnable loop
GitHub Copilot 13	Asked to avoid Roman-numbered pages in glossary references.	Explained how <code>\glsaddall</code> affects page references
GitHub Copilot 14	Asked to replace <code>iterrows</code> in the dataset loader with a better pattern.	Replaced <code>loop</code> with <code>Series.map</code> + boolean mask + <code>np.stack</code> for vectorized fingerprint loading
GitHub Copilot 15	Asked for help to deploy the quantized logP model onto an ESP32 with LiteRT.	Added a dedicated ESP-IDF project, a host-side Python serial script, and usage instructions for the ESP32 inference workflow
GitHub Copilot 16	Reported a serial port permission error while running the host SMILES sender against COM4.	Improved the Python script to catch serial open failures and print practical guidance about closing other serial monitors or processes
GitHub Copilot 17	Asked for a test pipeline that runs the dataset against the ESP32 logP model and produces charts	Added a resumable Python evaluation pipeline with subset selection modes, CSV summaries, and PDF plots
GitHub Copilot 18	Asked to reorganize the scripts for readability and maintainability.	Grouped imports, split constants into Paths and Hyperparameters sections, and reordered functions into Data, Model, Quantization and export, and Evaluation sections with separator comments
GitHub Copilot 19	Asked the ESP32 evaluator to retry failed molecules up to 100 times instead of moving on, and to treat device errors and timeouts as non-terminal so reruns retry them.	Added <code>MAX_RETRIES</code> constant, retry loop with serial buffer flush and re-PING between attempts, and changed <code>load_completed_indices</code> to only skip <code>ok</code> , <code>invalid_smiles</code> , and <code>max_retries_exceeded</code> results
GitHub Copilot 20	Reported that clamping persisted at 5.480893 after recalibration and asked how to eliminate it entirely.	Changed output type to float32 (TFLite inserts a DEQUANTIZE op; Dense layers still run int8) and updated the firmware to register <code>AddDequantize</code> , check for float32 output, and read <code>data.f[0]</code> directly.

System	Prompt	Usage
GitHub Copilot 21	Reported a boot loop after reflashing with the float32 output model.	Diagnosed peak arena overflow from the DEQUANTIZE op scratch buffers overrunning the heap during Invoke(); doubled kTensorArenaSize from 64 KB to 128 KB.
GitHub Copilot 22	Asked for a simple CLI script that takes a SMILES string and name, standardizes and canonicalizes it via the shared pre-processing module, generates a 2048-bit ECFP4 fingerprint, encodes the bit vector as a QR code, and saves it under the Results folder.	Added a Python CLI utility that writes QR images to Code/Results/qr_codes and installed the qrcode dependency for the active environment.
GitHub Copilot 23	Reported a QR generation failure and then asked to read the saved QR code back into a bit vector and throw an error if the round trip does not match.	Changed the QR payload to a compact hex-encoded fingerprint and added OpenCV-based QR decoding plus strict round-trip verification before reporting success.
GitHub Copilot 24	Asked for code to save a PDF graph of training/validation loss and MAE per epoch in Results/figures/history.pdf.	Added matplotlib code to plot and save training history after model training.

Declaration of Authorship

I hereby confirm that I have personally and independently prepared the present work and have not used any sources or aids other than those specified. All passages taken verbatim or in substance from other sources are identified as such. The drawings, illustrations and tables in this work are created by me or have been provided with an appropriate source reference. This work has not been submitted by me to any other university in the same or similar form for the acquisition of an academic degree.

Frankfurt, March 29, 2026.....

Rubin Chempananickal James